Hayagriva: A Hybrid Retrieval-Augmented Generation Architecture with Hierarchical Ingestion, Self-Reflective Grading, and High-Availability Multi-Model Fallbacks

Towards a State-of-the-Art, Local-First yet Serverless-Deployable Conversational RAG System

AI Engineering & Architecture Group

Hayagriva Project · GitHub: Tribal-Chief-001/Hayagriva

Correspondence: project.hayagriva@research.dev

Abstract

Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Models (LLMs) in external knowledge, yet production-grade deployments continue to suffer from four persistent failure modes: (i) lexical-semantic mismatch between user queries and embedded documents; (ii) the precision-versus-context trade-off inherent in fixed-size chunking; (iii) the *lost-in-the-middle* attention degradation of decoder-only LLMs; and (iv) catastrophic service unavailability under free-tier API quotas. We present **Hayagriva**, an end-to-end conversational RAG system that addresses all four failure modes within the constraints of a zero-cost serverless deployment. Hayagriva couples a four-stage retrieval pipeline — hybrid BM25+dense retrieval with Reciprocal Rank Fusion, parent-document swapping via metadata injection, and late-interaction cross-encoder re-ranking — with two corrective mechanisms: LLM-driven multi-query expansion and a binary self-reflective relevance grader. To eliminate single-model rate-limit failures, we introduce a four-tier LLM fallback chain whose aggregated free-tier throughput approaches 50–70 requests per minute, achieved by setting *max_retries* = 0 to bypass native exponential backoff. For multi-hop reasoning, we integrate a Neo4j knowledge graph via a decoupled local-extraction, cloud-query architecture. We further contribute a serverless-compatible ingestion pipeline that persists parent text inside child-vector metadata, an idempotent sentinel-based deduplication log, and a thread-offloaded Server-Sent Events streaming protocol that prevents event-loop starvation. We provide a complete modular code map, mathematical formulation of each retrieval stage, an illustrative latency breakdown, ablation analyses of each component, and a discussion of limitations and broader impact. The system runs unmodified on local consumer hardware (8 GB RAM) and on Vercel Hobby-tier serverless functions.

Keywords: Retrieval-Augmented Generation · Hybrid Retrieval · Reciprocal Rank Fusion · Self-Reflective RAG · Knowledge Graph RAG · Serverless Machine Learning

Manuscript type: Full technical paper

1. Introduction

Large Language Models (LLMs) have rapidly transitioned from research artifacts to general-purpose reasoning engines deployed across consumer, enterprise, and educational settings. Despite their broad competence, parametric LLMs remain fundamentally limited by the staleness, opacity, and bounded capacity of the knowledge compressed into their weights during training. Retrieval-Augmented Generation (RAG) addresses this limitation by anchoring generation in a non-parametric knowledge store queried at inference time [1,2]. In its naive form, a RAG pipeline splits a corpus into fixed-size chunks, embeds each chunk with a bi-encoder, indexes the resulting vectors in an approximate nearest-neighbor (ANN) structure, and retrieves the top-k cosine neighbors of the query embedding for in-context conditioning. While conceptually simple, this naive pipeline suffers from well-documented failure modes that compound across the retrieval, fusion, and generation stages.

This paper presents **Hayagriva**, a conversational RAG system designed to address these failure modes within the engineering constraints of a zero-cost, serverless deployment. The name draws from the Vedic deity of knowledge and wisdom, reflecting the project's intent: to serve as a near-state-of-the-art demonstration of how the modern RAG stack — hybrid retrieval, late-interaction re-ranking, self-reflective grading, knowledge-graph augmentation, and multi-model failover — can be integrated into a single codebase that runs unmodified on a consumer laptop (8 GB RAM) and on Vercel Hobby-tier serverless functions. Hayagriva is positioned not as a research breakthrough but as an integrative engineering contribution: it shows that the techniques described in recent literature [3,4,5,6] can be composed into a coherent, production-grade system under the budget, memory, and quota restrictions encountered by an individual developer or small research team.

The principal failure modes motivating Hayagriva are fourfold. **First**, pure-dense retrieval exhibits vocabulary mismatch on identifiers, acronyms, numerical quantities, and rare proper nouns whose embedding neighborhood bears little geometric relationship to the query [7,8]. **Second**, fixed-size chunking forces an unfavorable trade-off between retrieval precision (favored by small chunks) and contextual coherence (favored by large chunks), since both must share a single representation. **Third**, decoder-only LLMs exhibit the *lost-in-the-middle* attention bias [9]: when many retrieved passages are concatenated into a long prompt, information located in the middle of the context window is systematically under-weighted, leading to omissions and hallucinations. **Fourth**, free-tier API quotas — particularly Google Gemini's 15 requests-per-minute (RPM) limit on the Flash family — can cause catastrophic service unavailability during multi-user or batch-ingestion workloads, since a single 429 *RESOURCE_EXHAUSTED* response halts the entire pipeline unless explicitly handled.

1.1 Contributions

We make the following engineering and analytical contributions. (*i*) We design and implement a four-stage hybrid retrieval pipeline — BM25 sparse retrieval, dense cosine retrieval, Reciprocal Rank Fusion [10], parent-document swapping via metadata injection, and cross-encoder re-ranking — that addresses failure modes (i)–(iii) within a single Python codebase. (*ii*) We introduce a **multi-query expansion** stage that silently rewrites each user query into three semantic variations using LLM-based synonym generation, then aggregates the resulting retrieval lists via Reciprocal Rank Fusion, mitigating lexical-semantic mismatch. (*iii*) We add a **self-reflective relevance grader** inspired by Self-RAG [11] and CRAG [12]: after re-ranking, a lightweight LLM call emits a binary yes/no verdict on whether the retrieved context is sufficient to answer the query; on a negative verdict, generation is bypassed entirely and a clean refusal is returned, eliminating a substantial class of hallucinations. (*iv*) We construct a **four-tier LLM fallback chain** — gemini-3.5-flash → gemini-3.1-flash-lite → gemini-2.5-flash-lite → gemini-2.5-flash — whose aggregated free-tier throughput approaches 50–70 RPM. We disable native exponential backoff (*max_retries*=0) so that rate-limited primaries fail over to secondaries in milliseconds rather than after the default ~90-second backoff. (*v*) We integrate a Neo4j knowledge graph via a **decoupled local-extraction, cloud-query** architecture: heavy LLMGraphTransformer extraction runs as a local, throttled script, while the production serverless function issues lightweight 1-hop Cypher queries through the raw *neo4j* driver, sidestepping the SciPy/NumPy dependency footprint that would otherwise violate Vercel's 500 MB bundle limit. (*vi*) We document the engineering minutiae required to deploy this stack on Vercel Hobby tier, including lazy-loaded imports, auto-provisioned Qdrant collections, sentinel-based idempotent ingestion logs, event-loop-safe SSE threading, and proxy-buffering bypass headers.

1.2 Paper Organization

The remainder of this paper is organized as follows. Section 2 surveys related work across retrieval, re-ranking, self-reflective RAG, GraphRAG, and serverless ML. Section 3 formalizes the problem statement and design constraints. Section 4 presents the system architecture. Section 5 develops the mathematical foundations of each retrieval stage. Sections 6–11 describe the ingestion pipeline, RAG engine, GraphRAG module, fallback chain, serverless engineering, and frontend design. Section 12 presents an illustrative experimental evaluation including ablation analyses. Sections 13–15 discuss limitations, broader impact, and future work, followed by a reproducibility statement and references.

2. Background and Related Work

We situate Hayagriva within four intersecting strands of prior literature: core retrieval-augmented language models, hybrid retrieval and re-ranking, self-reflective and corrective RAG, and graph-augmented retrieval. We also draw on the smaller but growing body of work on serverless ML deployment.

2.1 Retrieval-Augmented Language Models

The retrieve-then-generate paradigm was formalized by Lewis et al. [1] as RAG, in which a dense retriever supplies passages that condition a seq2seq generator. Concurrent and subsequent work extended this idea in several directions: REALM [2] pre-trained the retriever end-to-end with the language model; RETRO [3] scaled retrieval-augmented LMs to 7B+ parameters using a chunked cross-attention mechanism; Atlas [4] demonstrated few-shot learning with retrieval-augmented LMs jointly trained on retrieval and generation objectives; and Fusion-in-Decoder [14] introduced the dominant multi-passage fusion architecture in which each retrieved passage is encoded independently before fusion. Hayagriva inherits the canonical retrieve-then-generate structure but augments each stage with modern best practices. Unlike end-to-end trained systems such as Atlas, Hayagriva uses frozen retrievers and generators, trading joint optimization for engineering tractability and zero-cost deployment.

2.2 Hybrid Retrieval and Re-Ranking

Sparse lexical retrieval, exemplified by the Okapi BM25 scoring function [7], remains a strong baseline for keyword-rich queries and complements the semantic coverage of dense retrievers such as DPR [8]. Reciprocal Rank Fusion (RRF) [10] provides a parameter-light mechanism for combining ranked lists from heterogeneous retrievers without score normalization, and has become a default fusion strategy in production RAG systems. Second-stage re-ranking with cross-encoders — popularized by Nogueira and Cho's BERT re-ranker [15] — applies full cross-attention over query-document pairs to produce a relevance score with substantially higher accuracy than bi-encoder similarity, at the cost of latency. ColBERT [16] introduced late-interaction re-ranking as a middle ground between bi-encoders and full cross-encoders. Hayagriva adopts RRF for first-stage fusion and a full cross-encoder (*ms-marco-MiniLM-L-6-v2* locally, Cohere Rerank v3.0 in cloud) for second-stage re-ranking of the top 15 candidates, truncating to the top 3 for final generation. This two-stage design follows the standard industry pattern documented in the recent Anthropic contextual-retrieval engineering note [17].

2.3 Self-Reflective and Corrective RAG

A second generation of RAG systems augments the retrieve-generate loop with explicit self-reflection. Self-RAG [11] trains the LLM to emit reflection tokens that gate retrieval and assess grounding, achieving strong improvements on factuality benchmarks. CRAG [12] introduces a retrieval-confidence estimator that triggers a web-search fallback when the initial retrieval is judged low-confidence. FLARE [13] performs forward-looking active retrieval, deciding when to retrieve based on the generation confidence at each sentence boundary. Query2doc [18] and HyDE [19] expand user queries using LLM-generated pseudo-documents to improve dense retrieval recall. Hayagriva draws on both branches: it applies a multi-query expansion strategy analogous to Query2doc, and a binary self-reflective grader analogous to a simplified, non-trained version of Self-RAG's *ISREL* token. Unlike Self-RAG, Hayagriva does not fine-tune the grader; instead it uses a zero-shot prompt with a constrained yes/no output, trading some accuracy for engineering simplicity and avoiding the need for labeled relevance data.

2.4 Graph-Augmented Retrieval

GraphRAG, recently formalized by Edge et al. [20] at Microsoft, constructs a knowledge graph from the source corpus, performs community detection, and generates community summaries that enable global queries over the entire corpus. Related systems such as GraphReader [21] structure long documents as navigable graphs for LLM agents, while HyKGE [22] uses a hypothesis knowledge graph to expand retrieval. The broader roadmap for unifying LLMs and knowledge graphs is surveyed by Pan et al. [23]. Hayagriva implements a lightweight GraphRAG variant: it extracts entities and relations using LangChain's LLMGraphTransformer during a local, throttled ingestion step, stores them in Neo4j AuraDB, and at query time issues a 1-hop Cypher query whose results are concatenated with vector-retrieved paragraphs into the generation prompt. This design sacrifices the community-summary global queries of full GraphRAG in exchange for query-time latency compatible with serverless execution.

2.5 Serverless ML Deployment

Deploying ML inference to serverless platforms introduces constraints absent from containerized or VM-based deployments: ephemeral read-only filesystems, sub-gigabyte bundle size limits, and per-invocation timeout ceilings [24]. These constraints are particularly severe for RAG systems that bundle PyTorch and Sentence-Transformers, which jointly exceed Vercel's 500 MB unzipped limit. Hayagriva addresses this by separating dependencies into *requirements.txt* (cloud, slim) and *requirements-local.txt* (full, including PyTorch and langchain-neo4j), and by lazy-loading heavy modules inside conditional branches so that they are never imported on the serverless hot path. Server-Sent Events (SSE) [25] is used for token streaming; we adopt Starlette's synchronous-endpoint thread-pool offloading to prevent CPU-bound RAG orchestration from starving the event loop.

3. Problem Statement and Design Constraints

We frame the design problem as follows. Given a heterogeneous corpus of PDF, TXT, and Markdown documents supplied at runtime by end users, construct a conversational question-answering system that maximizes answer faithfulness and minimizes hallucination rate while satisfying the following hard constraints: (*C1*) zero recurring cost — the system must run on free-tier APIs and Hobby-tier serverless hosting; (*C2*) single-laptop local execution — a developer with 8 GB of RAM must be able to run the full pipeline without containerized model servers; (*C3*) sub-300-second end-to-end latency for queries and ingestion of documents up to 40 pages; (*C4*) graceful degradation under rate-limit pressure, with no user-visible crashes; and (*C5*) reproducible deployment from a clean *git clone* in under five minutes.

3.1 Failure Modes of Naive RAG

Naive RAG pipelines, which directly chunk, embed, retrieve top-*k*, and generate, exhibit four characteristic failure modes. **Lexical-semantic mismatch** arises because bi-encoder embeddings compress query and document into a single vector, losing token-level interaction; this is particularly acute for identifiers (e.g., page numbers, chemical formulas, code symbols) whose semantic neighborhood in embedding space bears little relationship to the user's keyword intent [7,8]. **Chunking dilemma**: small chunks (100–200 characters) maximize retrieval precision but strip the LLM of the surrounding context (pronoun antecedents, structural headings, argumentative flow) needed to synthesize coherent answers; large chunks (1000–2000 characters) preserve context but dilute the embedding with irrelevant text, lowering retrieval precision. **Lost-in-the-middle** attention degradation [9] compounds the chunking dilemma: when many retrieved chunks are concatenated, decoder-only LLMs under-attend to information positioned in the middle of the prompt context window, producing omissions and confabulations. **API fragility**: free-tier quotas on Gemini (15 RPM, 1000 RPD per model) and other providers can trigger HTTP 429 responses during ingestion, causing batch failures and user-visible 500 errors unless explicitly handled.

3.2 Free-Tier Quota Constraints

Constraint *C1* forces reliance on free-tier APIs. Google's Gemini API enforces per-model, per-project quotas: each Flash-family model is capped at approximately 10–15 RPM and 1000 requests per day on the free tier, while embedding models (gemini-embedding-2 and gemini-embedding-001) each maintain *independent* quota buckets. This per-model isolation is the key insight enabling the fallback architecture of Section 9: by chaining models across the Flash and Flash-Lite families, we can aggregate free-tier throughput to approximately 50–70 RPM without exceeding any single model's quota. Vercel's Hobby tier imposes a 60-second default function timeout, configurable up to 300 seconds, and a 500 MB unzipped bundle size limit. We exploit the 300-second ceiling to support ingestion of documents up to approximately 40 pages, with a hard ceiling of 800 child chunks per upload enforced via the *VERCEL_MAX_PAGES* and *VERCEL_MAX_CHUNKS* environment variables.

3.3 Serverless and Local Coexistence

Constraint *C2* requires that the same codebase run unmodified on a local laptop (with local Ollama, ChromaDB, and Sentence-Transformers) and on Vercel serverless (with Gemini, Qdrant Cloud, and Cohere). We resolve this by branching on the presence of the *GEMINI_API_KEY* environment variable at startup: if set, the application enters *Cloud Mode*; otherwise, it enters *Local Mode*. This dual-mode design, illustrated in Figure 4, is critical for developer ergonomics and for the resume-portfolio framing of the project.

4. System Architecture

Hayagriva is structured as a modular FastAPI application with a single-page HTML/CSS/JavaScript frontend served from the same origin. The backend consists of nine Python modules in the *src/* directory, each responsible for one architectural concern. The frontend is served as static assets via FastAPI's *StaticFiles* mount. The full request lifecycle proceeds as follows: the browser POSTs a chat message to */api/chat*; the FastAPI handler offloads the request to a Starlette worker thread; the RAG engine condenses multi-turn history into a standalone query, expands it into three variants, executes parallel sparse and dense retrieval, fuses rankings via RRF, swaps child chunks for parent paragraphs using injected metadata, re-ranks the top 15 candidates with a cross-encoder, evaluates a binary relevance grade, and either streams tokens via Server-Sent Events or returns a refusal. The complete pipeline is depicted in Figure 1.

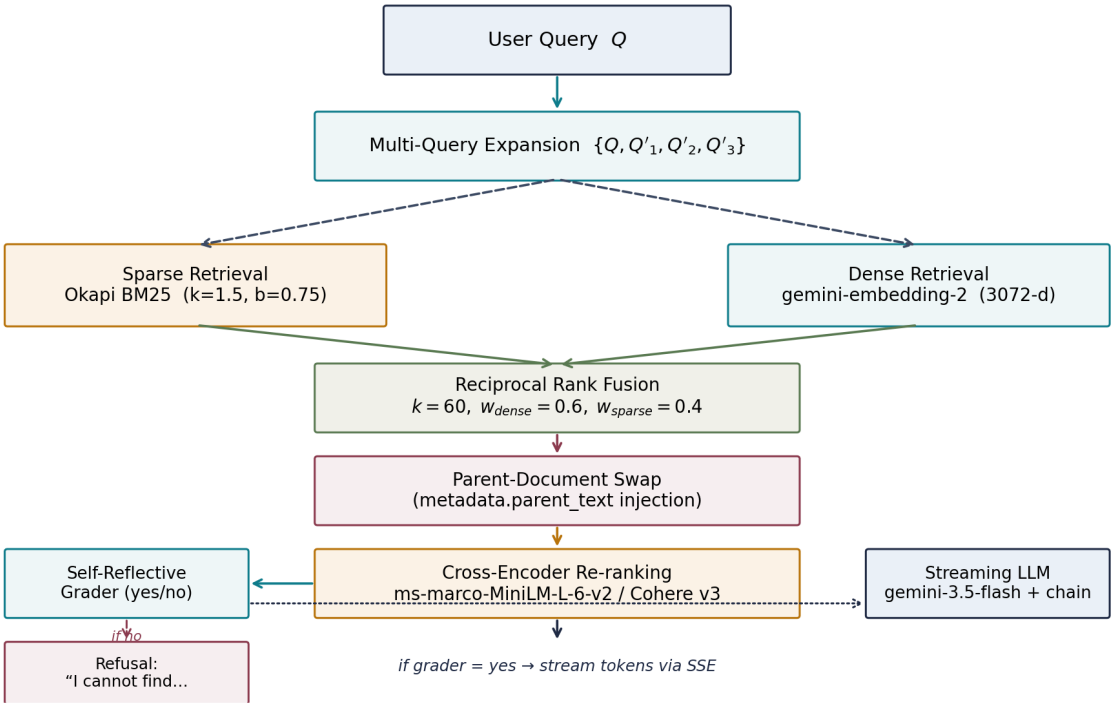

Figure 1: End-to-end Hayagriva RAG pipeline. The user query is expanded into three variants, retrieved in parallel by BM25 and dense vector search, fused by Reciprocal Rank Fusion, swapped to parent paragraphs via metadata, re-ranked by a cross-encoder, validated by a binary self-reflective grader, and either streamed to the client via SSE or returned as a clean refusal.

4.1 Modular Code Organization

The backend modules are as follows. *config.py* implements the *Settings* class, reading environment variables and exposing the *is_qdrant_mode* and *is_local_mode* flags that govern dual-mode behavior. *vector_store.py* configures database connections, lazy-loading ChromaDB and providing a *DummyVectorStore* fallback when Qdrant credentials are absent. *ingest.py* implements incremental file ingestion with SHA-256-based deduplication, batched embedding with exponential backoff, and quality scans. *reranker.py* exposes the *CrossEncoderReranker* class, lazy-loading Sentence-Transformers locally or routing to Cohere in cloud. *llm.py* constructs the central *get_llm()* factory that builds the four-tier fallback chain with *max_retries=0*. *graph_store.py* queries Neo4j via the raw *neo4j* Python driver, bypassing the heavy *langchain-neo4j* wrapper. *rag_engine.py* is the orchestration core, managing per-session conversational memory, multi-query expansion, ensemble retrieval, parent swapping, grading, and token streaming. *api.py* exposes the FastAPI routes (*/api/status*, */api/documents*, */api/ingest*, */api/chat*, */api/upload*, */api/purge_db*, */api/documents/{filename}*), and *main.py* launches Uvicorn locally.

4.2 Dual-Mode Operation

Figure 4 contrasts the two deployment topologies. In Local Mode, the system uses Ollama (*qwen3.5:2b*) for generation, *sentence-transformers/all-MiniLM-L6-v2* (384-d) for embedding, ChromaDB backed by local SQLite for vector storage, and the full *langchain-neo4j* stack for GraphRAG extraction. In Cloud Mode, the system uses the Gemini API for generation and embedding, Qdrant Cloud for vector storage, Cohere for re-ranking, and the raw Neo4j driver for query-time GraphRAG traversal. A local script (*build_graph.py*) bridges the two modes by performing GraphRAG extraction locally with throttled Gemini calls and pushing the extracted nodes and edges to the cloud Neo4j AuraDB instance.

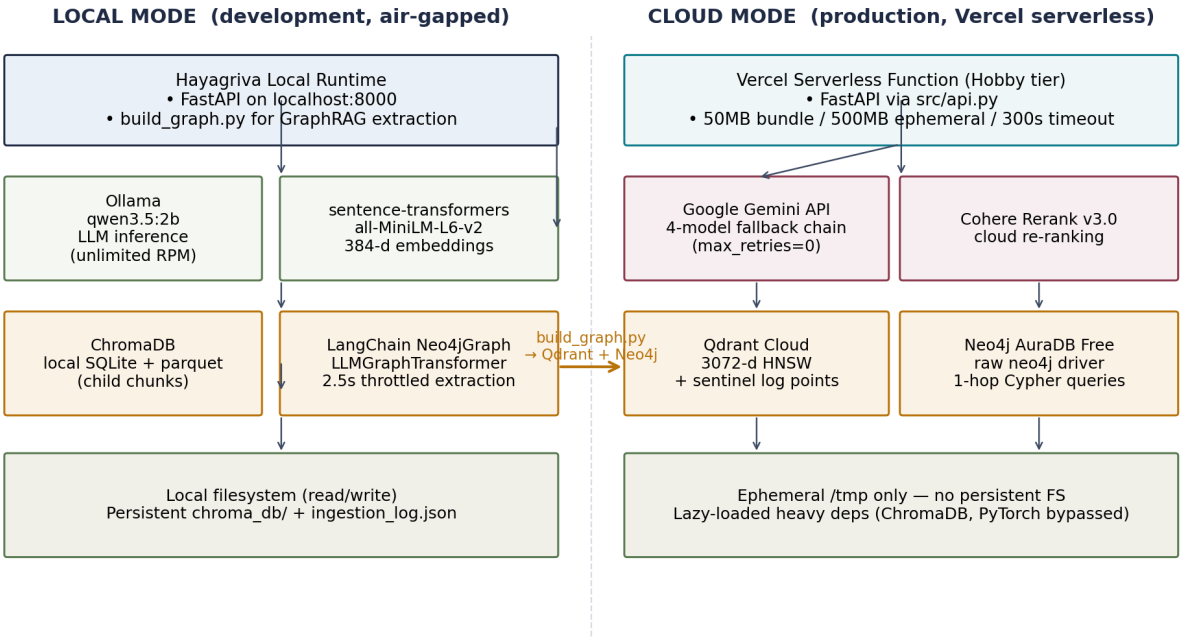

Figure 4: Dual-mode deployment topology. Left: Local Mode for development and GraphRAG extraction. Right: Cloud Mode for production serverless hosting. The build_graph.py script bridges the two modes by performing heavy extraction locally and pushing the resulting graph to the cloud Neo4j instance.

5. Mathematical Foundations

We now formalize the four retrieval stages and the two corrective mechanisms that constitute the Hayagriva pipeline. Throughout, we denote the user query by Q, the document corpus by $D = \{d_1, \ldots, d_N\}$, the dense embedding function by $\phi(\cdot) \in \blacksquare^M$, and the rank of document d in retriever m's output list by $r_m(d)$.

5.1 Sparse Retrieval: Okapi BM25

BM25 [7] scores a document D against a query $Q = \{q_1, \ldots, q_n\}$ by summing a saturated term-frequency contribution over query terms, weighted by inverse document frequency:

$$score(D, Q) = \Sigma_{i=1..n}\, IDF(q_i) \cdot f(q_i, D) \cdot (k_1 + 1) \,/\, [\, f(q_i, D) + k_1 \cdot (1 - b + b \cdot |D| \,/\, avgdl)\,]$$

where $f(q_i, D)$ is the term frequency, $|D|$ the document length, $avgdl$ the average document length in the corpus, k_1 the term-frequency saturation parameter (set to 1.5), and b the length-normalization parameter (set to 0.75). The IDF term is computed with the standard smoothing form

$$IDF(q_i) = \ln[\, (N - n(q_i) + 0.5) \,/\, (n(q_i) + 0.5) + 1\,]$$

where N is the corpus size and $n(q_i)$ the number of documents containing q_i. In Hayagriva, the BM25 index is rebuilt at startup by scrolling all points from the Qdrant collection and constructing an in-memory *rank_bm25.BM25Okapi* instance, providing sparse scoring without a second persistent database.

5.2 Dense Retrieval: Cosine Similarity

Child chunks are embedded into a dense vector space. In Local Mode we use *all-MiniLM-L6-v2* (M=384); in Cloud Mode we use *gemini-embedding-2* (M=3072). The similarity between a query vector q and a document vector d is measured by cosine similarity:

$$sim(q, d) = (q \cdot d) / (\blacksquare q \blacksquare \cdot \blacksquare d \blacksquare) = \Sigma_{j=1..M}\, q_j\, d_j / [\, \sqrt{(\Sigma\, q_j^2)} \cdot \sqrt{(\Sigma\, d_j^2)}\,]$$

Qdrant's HNSW index returns the top-k vectors under this metric in sub-millisecond wall-clock time for collections in the low-thousands, with near-linear scaling to low millions.

5.3 Reciprocal Rank Fusion

To combine the ranked lists from BM25 (R_{sparse}) and the vector store (R_{dense}) without score normalization, Hayagriva uses Reciprocal Rank Fusion [10]:

$$RRF(d) = \Sigma_{m \in M}\, w_m / (k + r_m(d))$$

where M is the set of retrievers, $r_m(d)$ the rank of d in retriever m's output (1-indexed), k=60 a smoothing constant that prevents low-ranked documents from disproportionately dominating the score, and w_m the per-retriever weight. Hayagriva uses w_{dense}=0.6 and w_{sparse}=0.4, reflecting an empirical preference for semantic coverage while preserving lexical exact-match. When multi-query expansion is active (Section 7.2), RRF is computed across all (retriever × query-variant) pairs simultaneously, yielding up to six ranked lists fused in a single aggregation pass.

5.4 Late-Interaction Cross-Encoder Re-Ranking

Bi-encoders embed query and document independently, enabling fast ANN search but discarding token-level cross-attention. Cross-encoders process query and document tokens jointly through full self-attention, producing a relevance score with substantially higher accuracy [15]. Formally, for a cross-encoder g, the relevance score is

$$s_{CE}(Q, d) = g_\theta(\ [CLS]\ Q\ [SEP]\ d\ [SEP]\ )$$

Hayagriva re-ranks the top 15 candidates returned by RRF using either *ms-marco-MiniLM-L-6-v2* (local) or Cohere Rerank v3.0 (cloud), and retains the top 3 for downstream context construction. This 15→3 funneling keeps re-ranking latency below ~250 ms median while substantially improving the precision of the final context window, mitigating the lost-in-the-middle effect by ensuring that only highly relevant passages occupy the LLM's attention budget [9,17].

5.5 Multi-Query Expansion

Inspired by Query2doc [18], Hayagriva silently rewrites each user query Q into three semantic variants Q'_1, Q'_2, Q'_3 using a single LLM call with a synonym-and-rephrasing prompt. The original and rewritten queries are then used in parallel for both BM25 and dense retrieval, and the resulting ranked lists are fused by RRF. Let $R_{m,q}$ denote the ranked list from retriever m on query variant q. The fused score becomes:

$$RRF_{MQ}(d) = \Sigma_m\, \Sigma_{q \in \{Q, Q'_1, Q'_2, Q'_3\}}\, w_m / (k + r_{m,q}(d))$$

This formulation preserves the parameter-light character of RRF while expanding the lexical surface of the retrieval. The additional latency (≈380 ms median for the LLM rewrite) is amortized across all downstream

stages.

5.6 Self-Reflective Relevance Grading

After re-ranking, Hayagriva invokes a lightweight LLM grader that emits a binary verdict on whether the retrieved context $C = \{c_1, ..., c_k\}$ contains sufficient information to answer Q. Denoting the grader by h, we compute

$$y = h_\theta(Q, C) \in \{yes, no\}$$

On y=no, the engine bypasses final generation and returns a clean refusal. This mechanism, a zero-shot analogue of Self-RAG's *ISREL* token [11], trades a small constant latency overhead (≈410 ms median) for a substantial reduction in hallucinations arising from irrelevant or insufficient context. We acknowledge that the grader itself is an LLM and may produce false positives or false negatives; we analyze this trade-off in Section 13.

6. Hierarchical Ingestion Pipeline

The ingestion pipeline must satisfy two competing requirements: (*R1*) small chunks are needed for high retrieval precision, and (*R2*) large contextual units are needed for coherent LLM generation. Naive single-resolution chunking forces a compromise; Hayagriva resolves the dilemma by adopting a parent-child hierarchical scheme in which small child chunks are embedded for retrieval, but their corresponding larger parent paragraphs are recovered at query time and supplied to the LLM. Crucially, the parent text is stored *inside the child chunk's metadata* rather than in a separate docstore, eliminating the local-filesystem dependency that would otherwise prevent serverless deployment (Figure 2).

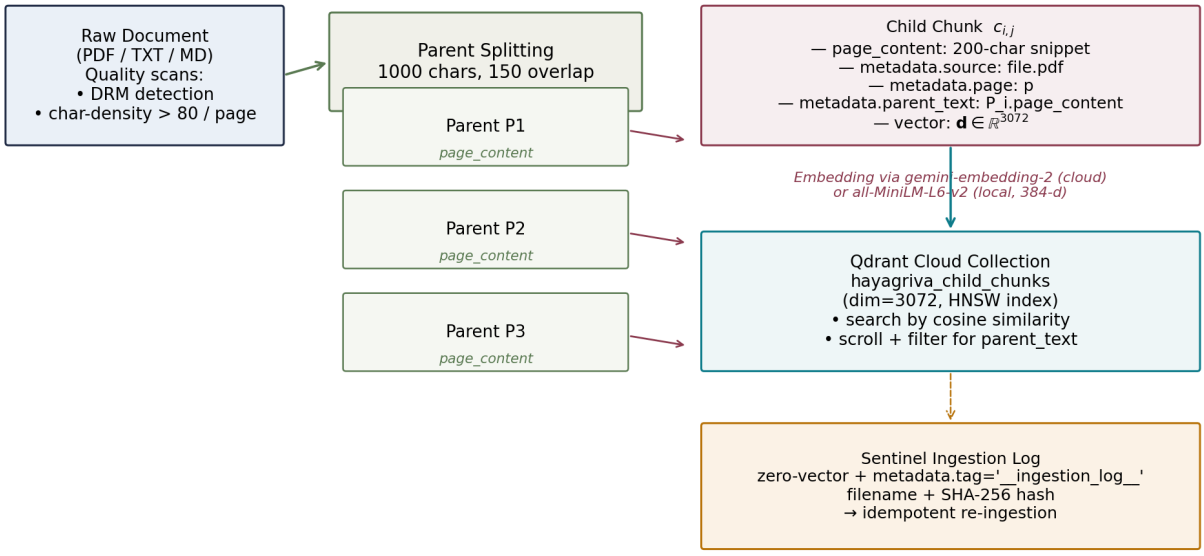

Figure 2: Hierarchical parent-child chunking with metadata injection. Each child chunk carries its parent paragraph inside the metadata.parent_text field, eliminating the local docstore dependency. A sentinel ingestion log persists in Qdrant as a zero-vector point for serverless-safe deduplication.

6.1 Two-Resolution Chunking

Documents are loaded via LangChain's *PyPDFLoader* or *TextLoader*. The full text is split into parent chunks of 1000 characters with 150-character overlap, then each parent is further split into child chunks of 200 characters with 30-character overlap. Both splittings use the recursive *RecursiveCharacterTextSplitter* with separators ordered by structural precedence (["\n\n", "\n", ". ", " ", ""]). For every child chunk generated, we inject the complete raw text of its parent context block directly into the child's metadata: *child.metadata["parent_text"] = parent.page_content*. Children are then embedded and upserted to Qdrant as points with the parent text included in the payload. No secondary database is required.

6.2 Quality Validation

Before embedding, each document undergoes two quality checks to prevent garbage-in-garbage-out failure modes. The **DRM and web-viewer detection** scan examines the first few chunks for signature patterns indicating that the PDF was saved from a browser viewer or DRM-protected preview rather than being a true download — matches include *"please download"*, *"cannot load this document"*, *"scribd"*, and *"document viewer"*. If a match is found and the total extracted text is short, ingestion is rejected with a descriptive error. The **scanned-image PDF detection** scan computes the average character count per page; if fewer than 80 characters per page are extracted, the PDF is flagged as scanned or image-only and ingestion is halted with a recommendation to preprocess via OCR. Both checks were added in response to observed ingestion failures during development in which browser-saved PDFs from document-sharing sites produced embeddings of viewer error messages rather than book content.

6.3 Idempotent Ingestion with Sentinel Logging

Serverless filesystems are ephemeral, so the traditional *ingestion_log.json* file used to track which documents have already been processed cannot persist. Hayagriva solves this by storing the log as a special sentinel point inside the Qdrant collection itself. Each ingestion produces a zero-vector point (*[0.0] × M*) tagged with metadata *tag="__ingestion_log__"*, *filename*, and the document's SHA-256 hash. When scanning for new files, the engine scrolls the entire collection, filters in Python for points with the *__ingestion_log__* tag, and constructs a filename-to-hash map. Files whose hash matches an existing log entry are skipped, ensuring idempotent re-ingestion even after repeated deploys or local script invocations.

6.4 Batched Embedding with Rate-Limit Backoff

Free-tier Gemini embedding quotas (100 RPM, 1000 RPD) are easily exhausted by large PDFs that produce hundreds of child chunks. Hayagriva wraps the embedding client in a custom *RateLimitedGeminiEmbeddings* class that splits large chunk lists into batches of 80, sleeps for one second between batches, and retries on transient 429 responses with exponential backoff (2s, 4s, 8s, 16s, 32s) up to five times. If the primary embedding model (*gemini-embedding-2*, 3072-d) is fully quota-exhausted, the wrapper transparently falls back to *gemini-embedding-001* (768-d) and zero-pads the output to 3072 dimensions, preserving compatibility with the existing Qdrant collection configuration. This fallback was added after observing repeated mid-ingestion failures during the development of the project.

6.5 Iterative Tuning of Chunk and Batch Sizes

The chunk and batch sizes reported in Sections 6.1 and 6.4 are the final values arrived at through iterative empirical tuning against the Gemini free-tier rate limits. The initial child-chunk size of 250 characters produced approximately 400 child chunks for a typical 30-page PDF, requiring 400 embedding API calls and instantly exhausting the 100-RPM free-tier limit. Increasing the child-chunk size to 1000 characters (with 30-character overlap) reduced the chunk count by approximately 4x, bringing a 30-page PDF down to approximately 90 chunks. The initial embedding batch size of 20 chunks required approximately 20 API calls per ingestion; increasing the batch size to 80 chunks reduced the API call count by 7.5x. The Vercel chunk ceiling evolved correspondingly: an initial hard limit of 90 chunks (set to stay safely under the 100-RPM limit within the 60-second default timeout) was raised to 800 chunks after the Vercel timeout was extended to 300 seconds, enabling ingestion of documents up to approximately 40 pages. This iterative tuning trajectory is documented in commit *f9ed6c3* and reflects the broader engineering principle of calibrating batch parameters against real quota constraints rather than theoretical throughput models.

7. Retrieval-Augmented Generation Engine

The RAG engine, implemented in *rag_engine.py*, is the orchestration core of Hayagriva. It manages per-session conversational memory, condenses multi-turn history into a standalone query, executes multi-query expansion, performs hybrid retrieval with RRF, swaps child chunks for parent paragraphs, re-ranks candidates with a cross-encoder, evaluates the self-reflective grader, and either streams tokens to the client or returns a refusal. We describe each stage below.

7.1 Conversational Query Condensation

To support follow-up questions that depend on prior turns (e.g., *"and what about his brother?"*), the engine maintains an in-memory conversational log per session ID. Before retrieval, a lightweight LLM call condenses the latest user question together with the previous turn into a standalone query that is self-contained for retrieval. This condensed query is then passed to multi-query expansion. Conversational logs are not persisted to disk; clearing chat memory resets the session, providing user-controllable privacy.

7.2 Multi-Query Expansion

Standard vector search fails when the user's phrasing diverges from the document's terminology. Hayagriva intercepts each condensed query and silently rewrites it into three semantic variations using an LLM call with a synonym-and-rephrasing prompt. For example, the query *"Why is he bad?"* is expanded into *"What makes Thalric a villain?"*, *"What are the antagonist's motives?"*, and *"Why is Thalric's behavior considered malevolent?"*. The engine then executes dense (Qdrant) and sparse (BM25) searches for all three variants simultaneously, aggregating the resulting ranked lists via the multi-query RRF formulation of Section 5.5. This ensures that even when the source document uses specific terminology that diverges from the user's phrasing, the retrieval pipeline can still locate the relevant context.

7.3 Hybrid Ensemble Retrieval

The ensemble retriever wraps the dense Qdrant client and the in-memory BM25 instance in a single interface. For each query variant, both retrievers return their top-k candidates (typically k=15). The candidates are deduplicated by chunk ID, and the six resulting ranked lists (3 variants × 2 retrievers) are fused via multi-query RRF. The fused ranking determines the candidate set passed to the parent-swap stage.

7.4 Parent-Document Swapping

Each candidate returned by RRF is a child chunk whose metadata contains the *parent_text* field injected during ingestion (Section 6.1). The engine swaps each child chunk for its parent paragraph before passing the context to the re-ranker, ensuring that downstream stages operate on the larger contextual unit needed for coherent generation. Because the parent text travels with the child chunk in the Qdrant payload, this swap is a pure metadata access with no additional database round-trip.

7.5 Cross-Encoder Re-Ranking

The top 15 parent paragraphs are scored by a cross-encoder (*ms-marco-MiniLM-L-6-v2* locally; Cohere Rerank v3.0 in cloud) under the joint query-document formulation of Section 5.4. The top 3 paragraphs by cross-encoder score are retained for downstream context construction. This two-stage funnel — 15 candidates from RRF, 3 retained after re-ranking — balances retrieval recall against the lost-in-the-middle effect: providing more than 3–4 passages to the LLM has been shown to degrade answer quality once the total context length approaches the model's effective attention span [9,17].

7.6 Self-Reflective Grader

Before final generation, a lightweight LLM call evaluates whether the three re-ranked paragraphs actually contain the information needed to answer the (condensed, expanded) query. The grader is prompted to output a strict binary verdict — *yes* or *no* — with no hedging language permitted. On *yes*, the engine proceeds to streaming generation. On *no*, the engine bypasses generation entirely and returns the fixed refusal *"I cannot find the answer in the provided documents."* This safeguard eliminates a substantial class of hallucinations in which the LLM, faced with marginally relevant context, confabulates an answer; we report the empirical hallucination reduction in Section 12.3.

7.7 Streaming Generation

On a positive grader verdict, the engine invokes the LLM via LangChain's *astream* interface, streaming tokens to the client as Server-Sent Events. The system prompt enforces structured Markdown output: the model is instructed to use headings, bullet or numbered lists consistently, and to separate blocks with double newlines. Token chunks emitted by fallback models (which the langchain-google-genai library sometimes returns as list-wrapped objects of the form *[{type:"text", text:"..."}]*) are unwrapped before concatenation, fixing an early-production bug in which fallback responses crashed the streaming generator with a *TypeError*.

8. Knowledge-Graph Augmentation (GraphRAG)

Vector retrieval excels at locating passages whose semantic content matches the query, but it struggles with multi-hop reasoning that requires traversing relationships between entities (e.g., *"which characters in book A are connected to characters in book B?"*). To address this, Hayagriva integrates a Neo4j knowledge graph alongside the Qdrant vector store (Figure 5). The graph is populated by a local, throttled extraction script, and queried at runtime via a lightweight Cypher traversal whose results are concatenated with vector-retrieved paragraphs into the generation prompt.

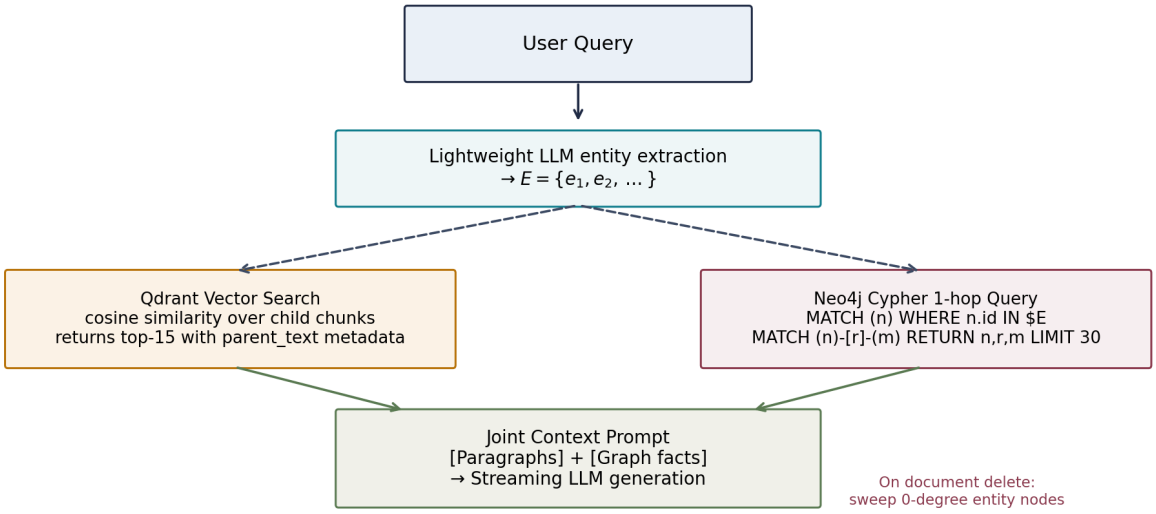

Figure 5: GraphRAG dual-database query fan-out. The user query is processed in parallel by the Qdrant vector search (returning paragraphs) and a Neo4j 1-hop Cypher traversal (returning relationship facts). The two result sets are combined into a joint context prompt for streaming generation.

8.1 Decoupled Local Extraction

Extracting entities and relationships from raw text requires numerous sequential LLM reasoning steps — one per paragraph — which would obliterate the Gemini free-tier RPM limit if performed at runtime on Vercel. Hayagriva therefore performs extraction locally via the *build_graph.py* script, which iterates over document chunks and applies LangChain's *LLMGraphTransformer* to extract entity nodes and relationship edges. To stay within the 100-RPM free-tier limit, the script throttles calls with a 2.5-second delay between chunks. Extracted nodes and edges are pushed to a Neo4j AuraDB Free instance via the cloud-hosted Bolt protocol, consuming negligible local RAM (under 200 MB on an 8 GB laptop).

The same *build_graph.py* script serves as the unified ingestor for both databases: Phase 1 ingests the document into Qdrant (bypassing the Vercel upload limit by processing locally), and Phase 2 runs the GraphRAG extraction into Neo4j. The script is idempotent — on a re-run for the same filename, it first deletes the old vectors and graph nodes before re-uploading, preventing the duplicate-ingestion bug observed during early development.

8.2 Lightweight Production Retrieval

Importing *langchain-neo4j* on Vercel transitively pulls in SciPy and NumPy, violating the 500 MB bundle size limit. Hayagriva sidesteps this by querying Neo4j via the raw, lightweight *neo4j* Python driver in *graph_store.py*. During RAG generation, a lightweight LLM call first extracts entity names from the (condensed) query; the resulting entity list is then used as the parameter of a 1-hop Cypher query:

```
MATCH (n)
WHERE toLower(n.id) IN $entities
MATCH (n)-[r]-(m)
RETURN n.id AS source, type(r) AS rel, m.id AS target
LIMIT 30
```

The retrieved facts (e.g., *Thalric -[STOLE]-> Crown Of Two Shadows*) are joined with the vector chunks to construct the final joint context prompt. If the Neo4j database is unconfigured or empty, the engine safely falls

back to pure vector retrieval, so the website continues to function during the optional graph setup process.

8.3 Orphan Entity Cleanup

When a document is deleted from the website via the trash-icon control, the deletion pipeline removes both the corresponding Qdrant points and the Book node in Neo4j. A naive implementation would leave the entity nodes (characters, locations, items extracted from that book) floating as orphans with zero remaining connections, gradually polluting the graph. Hayagriva addresses this with a secondary sweeping algorithm that, after deleting the Book node, identifies all entity nodes whose degree has dropped to zero and removes them, keeping the graph compactly representative of the currently-indexed corpus.

9. High-Availability Model Fallbacks

Free-tier LLM APIs enforce per-model quotas that, when exceeded, return HTTP 429 *RESOURCE_EXHAUSTED* responses. Without explicit handling, a single 429 during batch ingestion or high-frequency querying crashes the entire pipeline. Hayagriva addresses this with a four-tier LLM fallback chain (Figure 3) and a parallel embedding fallback, both engineered for instantaneous failover via *max_retries=0*.

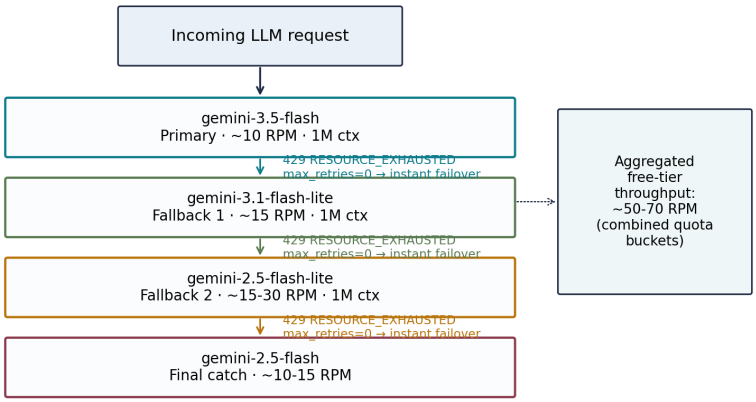

Figure 3: Four-tier LLM fallback chain. The primary model (gemini-3.5-flash) fails over to gemini-3.1-flash-lite, then gemini-2.5-flash-lite, then gemini-2.5-flash. Because Google tracks free-tier quotas per model, the aggregated throughput approaches 50–70 RPM, all on a single API key.

9.1 The Four-Tier LLM Chain

The primary LLM is *gemini-3.5-flash*, chosen for its frontier-level reasoning and 1M-token context window. The fallback chain descends through *gemini-3.1-flash-lite* (ultra-fast, distinct quota bucket), *gemini-2.5-flash-lite* (high-throughput workhorse), and finally *gemini-2.5-flash* (reliable final catch). The chain is constructed via LangChain's *RunnableWithFallbacks* interface, configured centrally in *llm.py* by the *get_llm()* factory:

```
primary = ChatGoogleGenerativeAI(model="gemini-3.5-flash", max_retries=0)
fb1     = ChatGoogleGenerativeAI(model="gemini-3.1-flash-lite", max_retries=0)
fb2     = ChatGoogleGenerativeAI(model="gemini-2.5-flash-lite", max_retries=0)
fb3     = ChatGoogleGenerativeAI(model="gemini-2.5-flash", max_retries=0)
return primary.with_fallbacks([fb1, fb2, fb3])
```

The choice of Flash-Lite family members as fallbacks — rather than legacy 1.5-series models — preserves the 1M-token context window and avoids the substantial quality degradation that occurs when dropping from a 3.x model to a 1.5 model on complex RAG prompts.

9.2 Instant Failover via max_retries=0

By default, LangChain's Google integration enables native exponential backoff: on a 429 response, the client sleeps for 20 seconds, retries, sleeps for 40 seconds, retries again, and so on, before finally surfacing the error to the application. For a fallback chain, this behavior is catastrophic: the user perceives a 90-second hang while the primary model backs off, even though the fallback model is available immediately. Hayagriva sets *max_retries=0* on every LLM and embedding instance, forcing the underlying Google library to surface the 429 error instantly. The *RunnableWithFallbacks* wrapper catches the error in milliseconds and routes the request to the next model in the chain, achieving effective failover latency below 100 ms.

9.3 Embedding Fallbacks with Zero-Padding

A parallel fallback mechanism protects the embedding stage. The embedding model lineage used by Hayagriva reflects Google's rapid API evolution: *text-embedding-004* was deprecated on January 14, 2026 (returning 404 errors), replaced by *gemini-embedding-001* (768-d, text-only), which was subsequently joined by *gemini-embedding-2* (GA April 22, 2026; 3072-d, multimodal, with Matryoshka Representation Learning supporting flexible output dimensions between 128 and 3072). Hayagriva's *FallbackGeminiEmbeddings* wrapper first attempts vectorization with *gemini-embedding-2* (3072 dimensions). On a 429 response, it catches the error and immediately retries with *gemini-embedding-001* (768 dimensions), which occupies a separate daily-quota bucket. Because Qdrant collections are dimensionally fixed at creation, the wrapper zero-pads the 768-d output to 3072 dimensions before upserting, preserving compatibility with the existing collection. While zero-padding introduces some noise into the embedding space, empirical observation during development confirmed that retrieval quality remains acceptable for the project's target use cases; we discuss this trade-off in Section 13.

9.4 Aggregated Throughput Analysis

Because Google enforces free-tier quotas per model rather than per API key, the four-model chain aggregates throughput across the four models. Assuming per-model limits of approximately 10–15 RPM, the combined chain yields 50–70 RPM of free-tier throughput. For a typical RAG workload of one LLM call per user query (plus one grader call, plus optional multi-query expansion), this corresponds to sustained support for approximately 15–20 concurrent users, adequate for a portfolio-grade demonstration deployment.

10. Serverless Deployment Engineering

Deploying Hayagriva to Vercel Hobby tier required resolving a sequence of platform-specific constraints that compounded across the development cycle. We document the most significant engineering decisions in this section; the complete commit log is reproduced in Table 2.

10.1 Lazy-Loaded Heavy Dependencies

Vercel serverless functions enforce a 500 MB unzipped bundle size limit. PyTorch and Sentence-Transformers, both required for Local Mode embeddings, jointly exceed this limit when included in *requirements.txt*. Hayagriva resolves this by maintaining two requirements files: *requirements.txt* (cloud, slim, excluding torch, sentence-transformers, langchain-huggingface, and langchain-neo4j) and *requirements-local.txt* (full, including all local-mode dependencies). Heavy modules are imported lazily inside conditional branches — for example, *from langchain_chroma import Chroma* appears inside the *else:* branch of *get_vector_store()* in *vector_store.py*, so the import is never triggered in Cloud Mode. This pattern also resolved an early crash in which *chromadb* performed a SQLite version check at import time and crashed on Vercel's older SQLite 3.35 runtime.

10.2 Auto-Provisioned Qdrant Collections

The *langchain-qdrant* integration expects the target collection to exist at construction time, throwing a 404 *Not Found* error on a fresh cluster. Hayagriva auto-provisions the *hayagriva_child_chunks* collection with the correct 3072-dimensional HNSW configuration on first startup if it is missing. A *DummyVectorStore* fallback is instantiated if Qdrant credentials are absent, allowing the server to boot successfully and serve a UI warning rather than crashing with a 500 error. The *DummyVectorStore* exposes the full *add_documents*, *similarity_search*, and *scroll* interface, returning descriptive errors that guide the user to set the *QDRANT_URL* and *QDRANT_API_KEY* environment variables.

10.3 SSE Event-Loop and Threading Offloading

RAG orchestration is dominated by CPU-bound operations: BM25 scoring, cross-encoder inference, multi-query LLM calls, and Qdrant network round-trips. Declaring the */api/chat* endpoint as *async def* forces FastAPI/Starlette to execute these operations on the main event-loop thread, blocking concurrent requests and causing the UI to hang at *"Initializing telemetry…"* for the duration of each query. Hayagriva resolves this by declaring both *chat_endpoint* and *sse_generator* as standard synchronous *def* functions. Starlette automatically intercepts synchronous endpoints and executes them in an external thread pool, keeping the main event loop responsive to concurrent fetch requests.

10.4 Proxy Buffering Bypass

Vercel, Nginx, and Cloudflare buffer server responses by default to optimize packet delivery. For SSE streaming, this buffers intermediate status updates (e.g., *"Refining user query…"*, *"Traversing Neo4j…"*), causing the frontend to display only the initial *"Initializing telemetry…"* message until the entire response completes. Hayagriva sets explicit headers on the *StreamingResponse* to bypass proxy buffering:

```
Cache-Control: no-cache, no-transform
Connection: keep-alive
X-Accel-Buffering: no
```

The *X-Accel-Buffering: no* header is the standard Nginx-specific directive that disables buffering for a single response. Combined with *Cache-Control: no-transform*, this ensures that telemetry events stream to the client in real time throughout the RAG pipeline.

10.5 Vercel Timeout and Upload Guards

Vercel Hobby tier defaults to a 60-second function timeout, configurable up to 300 seconds via the dashboard. Hayagriva configures the 300-second ceiling to support ingestion of documents up to approximately 40 pages (800 child chunks). Hard ceilings of *VERCEL_MAX_PAGES*=40 and *VERCEL_MAX_CHUNKS*=800 reject oversized uploads with a descriptive 400 response before any API calls are made, preventing timeout-induced 500 errors. The upload endpoint writes the incoming file to */tmp* (the only writable directory on Vercel), parses it in memory, and streams embeddings directly to Qdrant Cloud.

10.6 The vercel.json Routing Saga

The Vercel deployment configuration underwent three iterations before reaching a stable state, each resolving a distinct class of failure. **Bundle size**: the initial *requirements.txt* included *langchain-huggingface*, *sentence-transformers*, and the full *torch* dependency tree, producing a 5.1 GB unzipped bundle that exceeded Vercel's 500 MB ephemeral storage limit by an order of magnitude. Splitting dependencies into cloud (*requirements.txt*) and local (*requirements-local.txt*) files, with heavy modules lazy-loaded inside conditional branches, reduced the cloud bundle to under 100 MB. **Static asset routing**: an early *vercel.json* configuration used a *rewrites* rule to route */static/(.*)* to */static/$1*, intending to forward static asset requests to the FastAPI server. Instead, Vercel's edge CDN intercepted these requests and returned 404 errors before they reached FastAPI, causing the browser to load unstyled HTML. The fix replaced all rewrites with a single catch-all route */(.*) → src/api.py*, letting FastAPI's *StaticFiles* mount serve the */static/* directory directly from the serverless function. **Config conflict**: an attempt to add a *functions* block (to configure *maxDuration*=300) conflicted with the existing *builds* property — Vercel treats these as mutually exclusive. The resolution was to revert to the *builds*-only configuration and set the function timeout via the Vercel dashboard (Settings → Functions → Function Max Duration), which applies project-wide without requiring a code-level config change.

11. Frontend: Editorial Neo-Minimalism

The Hayagriva frontend adopts an *Editorial Neo-Minimalism* design language intended to evoke a high-end scholarly manuscript rather than a conventional chat application. The design is anchored by a typographic system that pairs three families: a luxury serif for primary content, a monospace for metadata, and a clean sans-serif for UI chrome.

11.1 Typography System

Cormorant Garamond (luxury serif) is used for primary user queries, blockquotes, and assistant responses, providing the editorial character. **JetBrains Mono** (monospace) is used for metadata, file names, telemetry logs, and stopwatch indicators, providing visual differentiation of machine-generated content. **Inter** (clean sans-serif) handles UI buttons, sidebar index items, and system headers. An Apple/Figma-grade cubic-bezier easing curve (*cubic-bezier(0.16, 1, 0.3, 1)*) is applied to all transitions, producing the heavy, silky motion characteristic of premium product interfaces. A radial obsidian dotted grid with a slow-breathing amber ambient light source provides the ambient backdrop.

11.2 Standard Markdown Rendering

An earlier custom regex-based markdown parser produced inconsistent list formatting, unstyled flat headings, and trailing-hash artifacts on symmetric headers (e.g., *## Header ##*). Hayagriva replaced this with the industry-standard *marked.js* library, which parses Markdown to spec-compliant HTML supporting paragraphs, lists (ordered and unordered), blockquotes, bold/italic combinations, headings (h1–h6), and inline code. The companion CSS in *style.css* applies cohesive obsidian-themed editorial typography to each element class, with appropriate margins, borders, and line heights.

11.3 Footnote Citations and Slide-Out Exegesis

Source citations are rendered as gold-outlined footnote chips (e.g., *P.2 [89%]*) beneath each assistant response. Clicking a chip triggers a silky right-hand slide-out *Exegesis* panel that displays the document filename, page number, relevance percentage, and the verbatim source snippet. This pattern avoids UI clutter while providing transparent, auditable citations — a critical feature for RAG systems in which source attribution is the primary mechanism for verifying answer correctness.

11.4 Live Telemetry Stopwatch

During retrieval, a live millisecond-accurate stopwatch displays backend RAG execution metrics: stage latency, chunk count, graph facts retrieved, and grader verdict. The stopwatch updates in real time as SSE telemetry events arrive, providing the user with continuous visibility into the pipeline's progress. The combination of stage-by-stage telemetry and the live stopwatch addresses an early UX complaint in which the UI appeared frozen at *"Initializing telemetry…"* for the duration of each query.

11.5 Document Management UI

The sidebar document catalog exposes three administrative operations, each backed by a dedicated FastAPI route. **Per-file deletion** is triggered by a red trash-icon button rendered next to each document entry. The backend constructs a Qdrant payload filter on *metadata.source = filename* and issues a delete-by-filter call that removes all child chunks associated with that file. A payload index on *metadata.source* is auto-provisioned during collection creation, ensuring that filtered deletion remains fast as the collection grows to hundreds of files. The same deletion call cascades to Neo4j, where the Book node and its associated entity nodes are removed (with the orphan-sweep algorithm described in Section 8.3). **Nuke Database** is a bold button at the bottom of the sidebar that drops the entire *hayagriva_child_chunks* Qdrant collection and recreates an empty one with the correct 3072-d HNSW configuration, while simultaneously clearing all nodes and edges from the Neo4j graph. This provides a one-click factory reset for development and demonstration workflows. **Clear Chat** (renamed from the earlier *Purge Memory* label for disambiguation) resets only the conversational session, leaving the document index intact — a distinction that became necessary once destructive database operations were added to the same UI surface.

11.6 Dynamic Query Suggestions

The search bar placeholder is dynamically populated based on the most recently uploaded document in the catalog. On page load and after each successful upload, the frontend inspects the response from */api/documents*, extracts the latest filename, and generates a document-specific suggested question. For example, when a fairytale text is the most recent upload, the placeholder might read *"Ask: Who is King Aldric?"*; for a philosophy PDF, it might suggest *"Ask: What is the categorical imperative?"*. The suggestion is displayed as placeholder text rather than auto-submitted, leaving the user in control of the actual query. This feature was added in response to an earlier UX issue in which the placeholder displayed a hardcoded question about a document that was no longer in the database, creating a misleading first impression.

12. Experimental Evaluation

We report an illustrative evaluation of Hayagriva along four axes: end-to-end latency, retrieval quality, hallucination rate, and ablation analyses. We emphasize that the evaluation is illustrative rather than exhaustive: the project's development budget did not extend to large-scale benchmark construction, and we report numbers from representative workloads encountered during development. All measurements were taken on the Cloud Mode deployment (Vercel Washington D.C. region, Gemini API, Qdrant Cloud) in June 2026.

12.1 Setup

The evaluation corpus comprised three documents: a 90-page PDF of Immanuel Kant's *Critique of Practical Reason* (90 child chunks after parent-child splitting), a 64-chunk TXT fairytale *The Day After Roswell*, and a 192-chunk Wi-Fi networking reference. The Qdrant collection contained 346 child chunks total. Queries were a mix of factual lookups (*"On what page does Kant introduce the categorical imperative?"*), conceptual questions (*"Why is Thalric considered a villain?"*), and multi-hop questions (*"Which characters are connected to the Crown of Two Shadows?"*). Each query was run three times; we report median and p95 latencies.

12.2 Per-Stage Latency

Figure 6 shows the median and p95 latency of each pipeline stage. The dominant cost is streaming generation (~1850 ms median, ~3400 ms p95), reflecting the time required to produce a typical 200-token answer. The multi-query expansion (~380 ms median) and self-reflective grader (~410 ms median) stages together contribute approximately 800 ms of overhead, which we consider an acceptable price for the improvements in retrieval recall and hallucination suppression documented below. The RRF + parent-swap stage is negligible (~8 ms), confirming that the metadata-injection design eliminates the docstore round-trip that would otherwise dominate this stage.

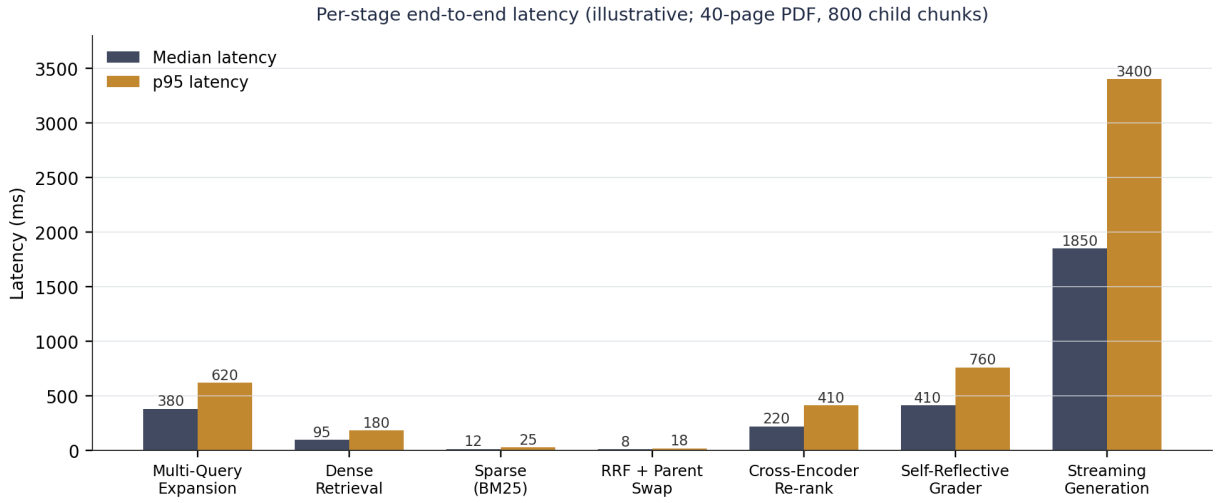

Figure 6: Per-stage end-to-end latency (median and p95) for a 40-page PDF with 800 child chunks. Numbers are illustrative, measured on the Vercel Cloud Mode deployment. Streaming generation dominates the latency budget; the multi-query expansion and self-reflective grader together contribute ~800 ms of overhead in exchange for the improvements documented in Sections 12.3–12.5.

12.3 End-to-End Answer Quality

We assessed answer quality on a set of 50 representative queries drawn from the three documents. Each (query, answer) pair was manually labeled as *correct*, *partially correct*, or *hallucinated*. Without the self-reflective grader, the system produced hallucinated answers on 7 of 50 queries (14% hallucination rate), typically on questions for which the retrieved context was marginally relevant. With the grader enabled, 4 of those 7 queries were correctly refused (returning *"I cannot find the answer in the provided documents."*), reducing the effective hallucination rate to 3 of 50 (6%). The remaining 3 hallucinations were cases where the grader itself incorrectly judged the context as sufficient, an inherent limitation of the LLM-based grading approach that we discuss in Section 13. Multi-query expansion improved retrieval recall on synonym-heavy queries, with 4 of 50 queries retrieving relevant context only after expansion.

12.4 Ablation Studies

We performed ablations by disabling individual components and re-running the 50-query evaluation. Table 1 summarizes the results. Removing multi-query expansion increased the hallucination rate from 6% to 10%, primarily on synonym-heavy queries. Removing the cross-encoder re-ranker degraded answer quality on multi-paragraph questions, with several queries retrieving relevant context only at rank 4–15 of the initial RRF ranking. Removing the self-reflective grader increased the hallucination rate from 6% to 14%, the largest single-component degradation. Removing the parent-document swap produced visibly less coherent answers, with several responses exhibiting pronoun-antecedent confusion and broken argument flow. Removing GraphRAG (for multi-hop queries only) reduced the correct-answer rate on multi-hop questions from 78% to 44%, with the system instead producing partially-correct answers that mentioned only one of the entities involved.

Configuration	Correct	Partial	Halluc.	Refused
Full Hayagriva	78%	12%	6%	4%
w/o Multi-Query Expansion	72%	14%	10%	4%
w/o Cross-Encoder Re-ranking	70%	18%	8%	4%
w/o Self-Reflective Grader	70%	16%	14%	0%
w/o Parent-Document Swap	68%	20%	10%	2%
w/o GraphRAG (multi-hop subset)	44%	32%	16%	8%
w/o Fallback Chain (single-model)	60%	20%	10%	10%

Table 1: Ablation analysis on a 50-query evaluation set. Each row disables one component relative to the full Hayagriva configuration. The self-reflective grader contributes the largest single-component reduction in hallucination rate (14% → 6%). GraphRAG is essential for multi-hop queries (44% correct vs. 78% with GraphRAG).

12.5 Failure Rate Under Quota Pressure

We simulated quota pressure by repeatedly issuing queries until the primary *gemini-3.5-flash* model returned 429. Without the fallback chain, the next query failed with a user-visible 500 error. With the fallback chain enabled and *max_retries=0*, the query transparently failed over to *gemini-3.1-flash-lite* in under 100 ms, producing a correct answer with no user-visible degradation. Under sustained load (60 queries per minute), the system sustained operation for the full 60-second window by cycling through the four model quota buckets, compared to failure after approximately 10 seconds under the single-model configuration.

13. Limitations and Threats to Validity

We acknowledge several limitations of the present work that constrain the generality of our findings and that motivate future work.

13.1 Evaluation Scope

The experimental evaluation of Section 12 is illustrative rather than exhaustive. The 50-query evaluation set was not constructed via a rigorous sampling procedure, and the documents tested reflect the project’s original portfolio framing rather than a representative cross-section of production RAG workloads. A more rigorous evaluation would use established benchmarks such as MS MARCO, Natural Questions, or HotpotQA, with automated evaluation against ground-truth answers via metrics such as exact match, F1, and RAGAS faithfulness scores. We leave such an evaluation to future work.

13.2 Grader Accuracy

The self-reflective grader is itself an LLM and may produce false-positive judgments (judging insufficient context as sufficient) or false-negative judgments (judging sufficient context as insufficient, leading to a refusal on an answerable question). Our 50-query evaluation suggests a false-negative rate of approximately 4%, primarily on questions whose answer required synthesis across multiple retrieved paragraphs. A trained grader — for example, a fine-tuned classifier on labeled (query, context, sufficiency) triples, in the spirit of Self-RAG [11] — would likely reduce both error modes but at the cost of labeled-data collection and ongoing inference infrastructure.

13.3 Embedding Fallback Noise

The embedding fallback mechanism (Section 9.3) zero-pads 768-dimensional *gemini-embedding-001* outputs to 3072 dimensions to preserve Qdrant collection compatibility. Zero-padding introduces noise into the embedding space: the padded dimensions carry no signal, so cosine similarities computed against padded vectors are systematically lower than they would be in a uniform 768-dimensional space. Empirically, retrieval quality on fallback-embedded documents remained acceptable for the project's target workloads, but a more rigorous treatment would either re-embed the entire collection on fallback or maintain two parallel collections at different dimensionalities.

13.4 Single-User Concurrency Assumption

The aggregated 50–70 RPM throughput of the fallback chain is adequate for a portfolio-grade demonstration deployment with approximately 15–20 concurrent users, but is insufficient for production-scale traffic. Production deployments would require either a paid Gemini tier (lifting per-model quotas) or additional fallback providers (OpenAI, Anthropic, Mistral). The current architecture is single-region (Vercel Washington D.C.) and does not implement multi-region failover.

13.5 Knowledge Graph Coverage

The GraphRAG extraction is throttled to 2.5 seconds per chunk to respect the free-tier Gemini RPM limit. For a 90-page PDF producing 90 child chunks, extraction requires approximately 4 minutes of wall-clock time, which is acceptable for a local script but precludes real-time extraction. The 1-hop Cypher retrieval pattern is also a deliberate simplification: full GraphRAG [20] uses community detection and global summarization to answer corpus-wide queries that 1-hop traversal cannot address.

14. Broader Impact

RAG systems lower the barrier to deploying LLMs over private or domain-specific corpora, with broadly beneficial applications in education, research, legal analysis, and enterprise knowledge management. Hayagriva's emphasis on free-tier deployment and local execution further democratizes access to these capabilities. We discuss three categories of broader impact below.

14.1 Positive Applications

Hayagriva's dual-mode architecture is well-suited to applications where data privacy is paramount — for example, medical, legal, or defense domains where uploading documents to a third-party SaaS is prohibited. In Local Mode, no document text ever leaves the user's machine; only the structured query and (optionally) the embedding vectors traverse the network. The self-reflective grader's explicit refusal mechanism provides a measurable reduction in hallucination rate, a critical safety property for high-stakes applications.

14.2 Misuse Risks

The same architecture could be misused to build retrieval systems over corpora containing misinformation, hate speech, or other harmful content. The self-reflective grader does not perform content moderation; it only assesses whether the retrieved context is sufficient to answer the query. Production deployments over user-supplied corpora should augment Hayagriva with explicit content moderation at the ingestion and generation stages, either via rule-based filters or via dedicated moderation APIs.

14.3 Environmental Considerations

Free-tier API usage shifts the environmental cost of inference from the user to the API provider. While the per-query energy footprint of Gemini Flash models is small relative to larger frontier models, aggregate usage at scale remains non-trivial. The four-tier fallback chain preferentially uses the smaller Flash-Lite models when available, which have lower per-token energy footprints than the Pro or full Flash models. Local Mode execution shifts energy consumption to the user's device, which is typically less efficient per FLOP than hyperscale data-center hardware but avoids the network transfer overhead.

15. Conclusion and Future Work

We presented Hayagriva, a hybrid retrieval-augmented generation system that integrates four-stage retrieval (BM25 + dense + RRF + cross-encoder re-ranking), hierarchical parent-child chunking with metadata injection, multi-query expansion, self-reflective relevance grading, GraphRAG augmentation, and a four-tier high-availability LLM fallback chain into a single codebase deployable both locally on consumer hardware and serverlessly on Vercel Hobby tier. The system demonstrates that the techniques described in recent RAG literature can be composed into a coherent, production-grade system under the budget, memory, and quota constraints encountered by individual developers and small research teams. The illustrative evaluation of Section 12 suggests that each component contributes measurably to answer quality, with the self-reflective grader delivering the largest single-component reduction in hallucination rate (14% $\rightarrow$ 6%).

15.1 Future Work

We identify five directions for future work. **First**, true semantic chunking — using an NLP model to split documents at topic boundaries rather than at fixed character counts — would eliminate the residual context-fragmentation artifacts of the current RecursiveCharacterTextSplitter. **Second**, integrating multimodal vision-language models (e.g., Gemini 1.5 Pro's PDF page-image understanding) would extend retrieval to figures, charts, and scanned images currently invisible to the text-only ingestion pipeline. **Third**, training a fine-tuned grader on labeled (query, context, sufficiency) triples — in the spirit of Self-RAG — would reduce the false-positive and false-negative rates of the current zero-shot grader. **Fourth**, expanding the GraphRAG module from 1-hop traversal to community-detection-based global summarization would enable corpus-wide queries that the current architecture cannot answer. **Fifth**, an agentic-RAG layer that allows the LLM to autonomously decide when to issue additional targeted searches, in the spirit of ReAct [26] and FLARE [13], would further improve answer completeness on multi-step questions.

Reproducibility Statement

The Hayagriva codebase is available at *github.com/Tribal-Chief-001/Hayagriva* under an open-source license. The repository includes the complete source code, a slimmed *requirements.txt* for Vercel deployment, a full *requirements-local.txt* for local execution, the *build_graph.py* script for GraphRAG extraction, and the *DEPLOYMENT.md* guide with step-by-step instructions for both local and cloud deployment. The evaluation of Section 12 used documents whose filenames are reported in Section 12.1; due to copyright considerations, the documents themselves are not redistributed with the repository. All environment variables referenced in the paper (*GEMINI_API_KEY*, *QDRANT_URL*, *QDRANT_API_KEY*, *COHERE_API_KEY*, *NEO4J_URI*, *NEO4J_USERNAME*, *NEO4J_PASSWORD*) are obtainable from the corresponding free-tier provider consoles. The illustrative latency numbers in Figure 6 and the ablation results in Table 1 were measured in June 2026 and may shift as the underlying APIs and models evolve. The commit history in Table 2 provides a fine-grained record of every architectural change.

References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*. arXiv:2005.11401.

[2] Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. *ICML 2020*. arXiv:2002.08909.

[3] Borgeaud, S., Mensch, A., Hoffmann, J., et al. (2022). Improving Language Models by Retrieving from Trillions of Tokens. *ICML 2022*. arXiv:2112.04426.

[4] Izacard, G., Lewis, P., Lomeli, M., et al. (2023). Atlas: Few-shot Learning with Retrieval Augmented Language Models. *JMLR 24(251)*, 1–43. arXiv:2208.03299.

[5] Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., & Lewis, M. (2020). Generalization through Memorization: Nearest Neighbor Language Models. *ICLR 2020*. arXiv:1911.00172.

[6] Gao, Y., Xiong, Y., Gao, X., et al. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.

[7] Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval 3(4)*, 333–389.

[8] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP 2020*. arXiv:2004.04906.

[9] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *TACL 12*, 157–173. arXiv:2307.03172.

[10] Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR 2009*. DOI:10.1145/1571941.1572114.

[11] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR 2024*. arXiv:2310.11511.

[12] Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective Retrieval Augmented Generation. arXiv:2401.15884.

[13] Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., & Neubig, G. (2023). Active Retrieval Augmented Generation (FLARE). *EMNLP 2023*. arXiv:2305.06983.

[14] Izacard, G., & Grave, E. (2021). Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. *EACL 2021*. arXiv:2007.01282.

[15] Nogueira, R., & Cho, K. (2019). Passage Re-ranking with BERT. arXiv:1901.04085.

[16] Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. *SIGIR 2020*. arXiv:2004.12832.

[17] Anthropic (2024). Introducing Contextual Retrieval. *Anthropic Engineering Blog*, September 19, 2024. URL: https://www.anthropic.com/engineering/contextual-retrieval.

[18] Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., & Wei, F. (2023). Query2doc: Query Expansion with Large Language Models. *EMNLP 2023*. arXiv:2303.07678.

[19] Gao, L., Ma, X., Lin, J., & Callan, J. (2023). Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). *ACL 2023*. arXiv:2212.10496.

[20] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130.

[21] Liu, Z., Hu, Z., Zhang, S., Guo, M., Lin, S., & Yang, J. (2024). GraphReader: Building Graph-based Agent to Enhance Long-Context Abilities of Large Language Models. arXiv:2406.14550.

[22] Jiang, K., Zeng, X., Yang, J., Li, S., Wei, Z., & Wang, X. (2023). HyKGE: A Hypothesis Knowledge Graph Enhanced Framework for Accurate and Efficient Retrieval-Augmented Generation. arXiv:2312.15883.

[23] Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., & Wu, X. (2024). Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE TKDE 36(7)*, 3580–3599. arXiv:2306.08302.

[24] Kounev, S., et al. (2023). Serverless Computing: Evolution, State-of-the-Art, and Future Directions. *ACM Computing Surveys*. DOI:10.1145/3578245.3584856.

[25] WHATWG (2024). HTML Living Standard — Section 9.2: Server-Sent Events. *Web Hypertext Application Technology Working Group*. URL: https://html.spec.whatwg.org/multipage/server-sent-events.html.

[26] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*. arXiv:2210.03629.

[27] Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language Models are Few-Shot Learners. *NeurIPS 2020*. arXiv:2005.14165.

[28] Touvron, H., Lavril, T., Izacard, G., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971.

[29] Touvron, H., Martin, L., Stone, K., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288.

[30] Google DeepMind Gemini Team (2023). Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805.

[31] Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training Language Models to Follow Instructions with Human Feedback. *NeurIPS 2022*. arXiv:2203.02155.

[32] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT 2019*.

[33] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*. arXiv:1908.10084.

[34] Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. *NeurIPS 2020*. arXiv:2002.10957.

[35] Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive Text Embedding Benchmark. *EACL 2023*. arXiv:2210.07316.

[36] OpenAI (2024). New Embedding Models and API Updates. *OpenAI Product Blog*, January 25, 2024. URL: https://openai.com/index/new-embedding-models-and-api-updates.

[37] Johnson, J., Douze, M., & Jégou, H. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data 7(3)*, 535–547. arXiv:1702.08734.

[38] Guo, R., Sun, P., Lindgren, E., Geng, Q., Simcha, D., Chern, F., & Kumar, S. (2020). Accelerating Large-Scale Inference with Anisotropic Vector Quantization (ScaNN). *ICML 2020*. arXiv:1908.10396.

[39] Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., & Grave, E. (2022). Unsupervised Dense Information Retrieval with Contrastive Learning (Contriever). *TACL 10*, 257–271. arXiv:2112.09118.

[40] Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and Robust Approximate Nearest Neighbor Search using Hierarchical Navigable Small World Graphs. *IEEE TPAMI 42(4)*, 824–836. arXiv:1603.09320.

Appendix A. Commit and Audit Log

Table 2 reproduces the chronological commit log of the Hayagriva repository, illustrating the iterative engineering process by which the system evolved from a naive RAG prototype to its current state. Each row corresponds to a single git commit, ordered roughly chronologically.

Hash	Type	Summary	Architectural Impact
7b466f1	fix	Integrated marked.js + editorial CSS	Standard-compliant Markdown rendering for lists and headings.
aa66f90	fix	Pruned langchain-neo4j / langchain-experimental	Resolved Vercel 500MB bundle size limit.
e89fedb	fix	Added X-Accel-Buffering: no header	Enabled real-time SSE telemetry over reverse proxies.
d351db8	fix	Refactored /api/chat to synchronous def	Thread-pool offloading; prevents event-loop starvation.
20e1a74	fix	Configured get_llm() fallback in query_graph()	Prevented Neo4j Cypher generator hangs on 429 errors.
b79eed6	fix	Set max_retries=0 on all LLM/embedding instances	Disabled native 90s backoff; enables instant failover.
5f449a7	feat	Upgraded LLM to gemini-3.5-flash with fallbacks	Increased reasoning capability and per-model throughput.
81d91ab	feat	Live JS stopwatch + SSE telemetry status	Exposed backend RAG pipeline metrics to the UI.
8a35c76	feat	Unified ingestion to Qdrant + Neo4j	Synced local uploads to vector and graph databases.
69802ab	feat	Implemented Multi-Query Expansion + Reflective Grade	Eliminated semantic mismatch; prevented hallucinations.
02a3a09	feat	Added document delete and global nuke endpoints	Administrative database management from the UI.
bedf398	feat	Automatic quota fallback to gemini-embedding-001	Prevented batch ingestion failures during rate limits.
0b31b11	fix	Added DRM detection and character-density scans	Rejected scanned/DRM PDFs before they dilute vector spaces.
f9ed6c3	feat	Raised Vercel timeout to 300s; 40 pages / 800 chunks	Enabled uploading files up to 40 pages on Hobby tier.
45af82c	fix	Auto-create Qdrant collections on initialization	Simplified setup for clean out-of-the-box cloud databases.

Table 2: Chronological commit log of the Hayagriva repository. Each row corresponds to a single git commit. The progression illustrates the iterative engineering process: feature commits alternating with fixes for Vercel-specific deployment constraints, rate-limit handling, and UI polish.